doccat Quick Start Guide

Oct 19, 2026

doccat (`doc_catalog_mcp`) is an MCP server that lets an agent work through thousands of PDF/Office documents using an **LLM-written metadata catalog** instead of a vector store — no embeddings, no chunking, no similarity search. The agent reads what each document *is about* and decides which ones to open, the way an analyst uses a library catalogue. This guide walks through install → configure → index → serve → connect, the fastest path to a working server. Source: doc_catalog_mcp on GitLab.

Install

Use a virtualenv, not a shared interpreter — the server needs `mcp>=1.9,<2` and mcpo is sensitive to the `starlette` / `fastapi` versions around it.

```
python -m venv .venv && source .venv/bin/activate
pip install -e .
```

This installs the `doccat` command. `make dev` adds the test dependencies; `pip install -e '.[proxy]'` adds mcpo for Open WebUI.

Supported inputs: PDF, DOCX, PPTX, XLSX, Markdown, TXT, HTML — format is detected from magic bytes, not the extension.

Step 1 — Configure once

```
doccat config init
```

Edit the `doccat.yaml` it writes — corpus directory, catalog path, LLM endpoint, worker count — then check what it resolves to:

```
doccat config show
```

Every command below then works with no flags. Flags and environment variables still override the file when needed, and skipping this step is fine too — pass flags instead.

Step 2 — Build the catalog

Point it at your documents and at any OpenAI-compatible endpoint (vLLM, llama.cpp, TGI, Ollama, LM Studio, the Hugging Face router):

```
doccat index --root /data/documents --catalog ./catalog.jsonl --base-url
http://localhost:8080/v1 --model qwen2.5-14b-instruct --llm-workers 16
```

See what profile it would pick for each document, without spending a token:

```
doccat index --root /data/documents --dry-run
```

Written next to the catalog:

Output	What it holds
`extracted_text/`	a Markdown copy of every extraction, with YAML front matter
`llm_errors/`	raw model replies that could not be parsed, for diagnosis
`needs_ocr.txt`	PDFs with no text layer
`needs_conversion.txt`	legacy binary Office files (`.doc/.ppt/.xls`); auto-converted and indexed if `soffice` (LibreOffice) is installed

Re-run the same command any time — unchanged files are skipped by content hash, new/modified files are indexed, deleted files are removed, and errored records are retried. `--force` rebuilds everything.

JSON dumps (e.g. session transcripts, tickets, articles as a JSON array with a text field) go through the same pipeline:

```
doccat json --input Plenos.json --field texto --catalog ./plenos.jsonl --
base-url http://localhost:8080/v1 --model my-model -j 16
```

Every other field is preserved under `source_metadata` and can drive profile routing (`match: {metadata_equals: {orga: Senado}}`).

Hugging Face inference router:

```
doccat index --root /data/documents --base-url
https://router.huggingface.co/v1 --model Qwen/Qwen3.6-35B-A3B --api-key
$HF_TOKEN --bill-to BSC-LT
```

Step 3 — Customize the cataloguing (profiles)

A cataloguing **profile** adapts the prompt, metadata schema and reading strategy to the domain: a contract gets `parties`, `expiry_date`, `risk_flags`; a session transcript gets `chamber`, `speakers`, `agenda_items`; a manual gets `systems`, `error_codes`.

```
doccat profiles list
doccat profiles show legal
doccat profiles init mydomain
```

Nine profiles ship with the package: `generic`, `legal`, `financial`, `parliamentary`, `technical`, `academic`, `intranet`, `minutes`, `long_document` (map-reduce for very long texts). Write your own as YAML, inherit from one of these stating only the differences, and load it with `--profiles-dir`:

```
doccat index --root /data/trials --profiles-dir ./profiles
```

Let an agent design the profile for you. On a repository nobody has catalogued before, `doccat optimize` reads a stratified sample, works out what the corpus is, drafts a profile, catalogues a few real documents with it, and revises based on what came back empty or useless:

```
doccat optimize --root /data/documents -o profiles/mydomain.yaml
```

Review the resulting YAML (it carries its own reasoning in comments), check routing with `--dry-run`, then index.

Step 4 — Scale it

```
doccat index --root /data/documents -j 32 --extract-workers 8 --rps 20
```

`--llm-workers` / `-j` sets concurrent LLM requests, `--extract-workers` the extraction process pool. A 429 from any worker pauses the whole pool; progress, rate and ETA are logged as the run proceeds; Ctrl-C drains in-flight work and saves.

For very large corpora, split across machines and merge:

```
doccat index --root /data/documents --catalog shard_0.jsonl --shard 0/4
doccat merge shard_*.jsonl -o catalog.jsonl
```

Step 5 — Build the corpus graph (optional)

If any profile used above has a `graph:` block (`legal` and `parliamentary` both do), the catalog already carries `relations` between entities. Resolve them across the whole corpus:

```
doccat graph build --catalog ./catalog.jsonl --summarize
```

This is what answers "how are X and Y connected?" questions, where keyword search alone comes up short.

Step 6 — Run the MCP server

```
doccat serve --docs-root /data/documents --catalog ./catalog.jsonl --port
8000
```

It listens at `http://<host>:8000/mcp` (streamable HTTP), serves a plain-HTTP health check at `/health`, and reloads the catalog automatically when it changes on disk. `--catalog` may be repeated, comma-separated, or a glob, so index shards can be served without merging them first.

With containers:

```
cp .env.example .env && docker compose -f deploy/docker-compose.yml --profile
index run --rm indexer
docker compose -f deploy/docker-compose.yml up -d server
```

On a systemd host: `sudo ./deploy/install.sh` — full instructions in `DEPLOYMENT.md`.

Step 7 — Connect an agent

```
{
  "mcpServers": {
    "doc_catalog": { "type": "http", "url": "http://localhost:8000/mcp" }
  }
}
```

Quick test with the MCP Inspector:

```
npx @modelcontextprotocol/inspector
```

Connect it to `http://localhost:8000/mcp` and call `catalog_overview`.

Tools exposed

Tool	Purpose
`catalog_overview`	Corpus stats: counts, doc types, top topics, profiles in use and the extra fields each one produced
`search_documents`	Ranked keyword search over all metadata, including profile-specific fields; filters by `doc_type`, `topic`, `profile`
`search_by_field`	Query one metadata field directly — "which contracts involve ACME?", "which manuals mention error E-403?"
`list_documents`	Browse by folder prefix, doc type or profile
`get_document_metadata`	The full record for one document
`read_document`	Extract the actual text on demand (PDF page ranges); falls back to the `extracted_text/` cache
`graph_overview` / `graph_search` / `graph_neighbors` / `graph_path` / `graph_communities`	Traverse the corpus graph built by `doccat graph build`

Tests

```
make test
```

The suite runs the whole pipeline against a mock OpenAI-compatible endpoint: configuration precedence, profile routing and inheritance, relation extraction, graph consolidation, the optimizer's sampling, parallel execution, rate-limit back-off, crash recovery, sharding, and the server's tools over a real MCP client.

Further reading

Doc	Covers
`DESIGN.md`	Rationale and architecture of the catalog-over-vector-store approach

Doc	Covers
`docs/CONFIGURATION.md`	The full config model — file, env, flags
`docs/PROFILES.md`	Every profile key, routing rules, excerpt strategies
`docs/OPTIMIZE.md`	How the profile-designing agent works
`docs/SCALING.md`	Worker sizing, throughput, resumability, sharding
`docs/GRAPH.md`	Corpus graph build and the graph MCP tools
`DEPLOYMENT.md`	Docker, systemd, and shared-environment repair